POLYNOMIAL INTERPOLATION

* Key task in data science is the construction of models from data.

DATA	→	MODEL
(x_i, y_i)		$y = g(x)$

↳ choose model class e.g. polynomial.

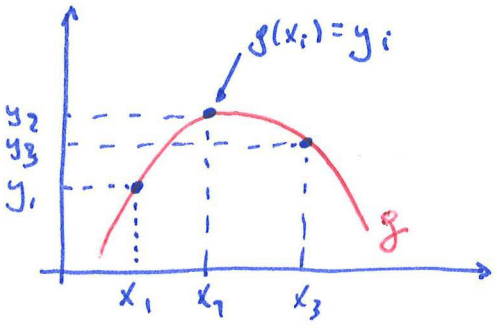
* DATA INTERPOLATION (T3)

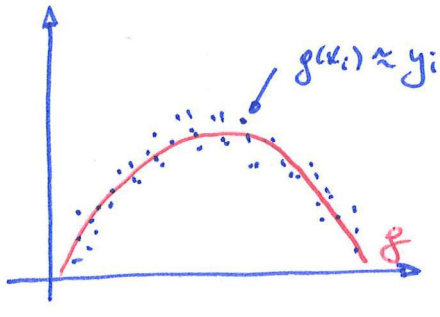
"Find g such that $g(x_i) = y_i$"

DATA APPROXIMATION. (T5)

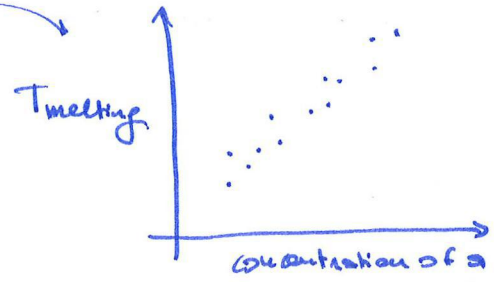
"Find g such that $g(x_i) \approx y_i$"

this requires a criterion to measure the misfit. e.g. LS.





* WHY? (First, ask students)
  - ↳ Intermediate values
  - ↳ Need an approx of derivative / integral of the underlying func (see T4) + T2: multi-step
  - ↳ Need smooth / continuous repr of the variables.
  - ↳ Motion planning — waypoints / robot arm (smooth motion)



Every experiment cost 10k €. So, let's do a few exps and interpolate intermediate points.

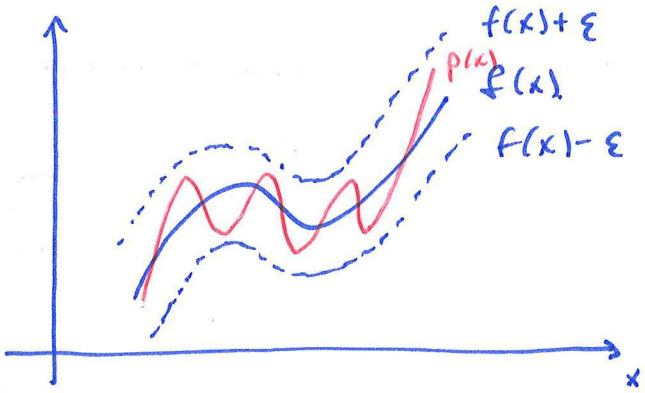
* Can we even do it? Is it possible?

↳ Practically, we often assume that relationships between variables in a physical problem can be approximately reproduced from data.

↳ From a mathematical pov, we have the Weierstrass Approx thm.

"Suppose f exists and is continuous on $[a,b]$. For each $\varepsilon > 0$, there exists a polynomial $p(x)$ defined on $[a,b]$, such that.

$$|f(x) - p(x)| < \varepsilon \quad \text{for all } x \in [a,b]."$$



$$\forall \varepsilon > 0, \exists p(x) : \forall x \in [a,b] : |f(x) - p(x)| < \varepsilon.$$

* Why polynomials? (First, ask students.)

- Easy to evaluate (eg. Horner, see T1)
- Derivs & Ints are also polynomials AND easy to compute.

(. And we have Weierstrass.)

* Overview.

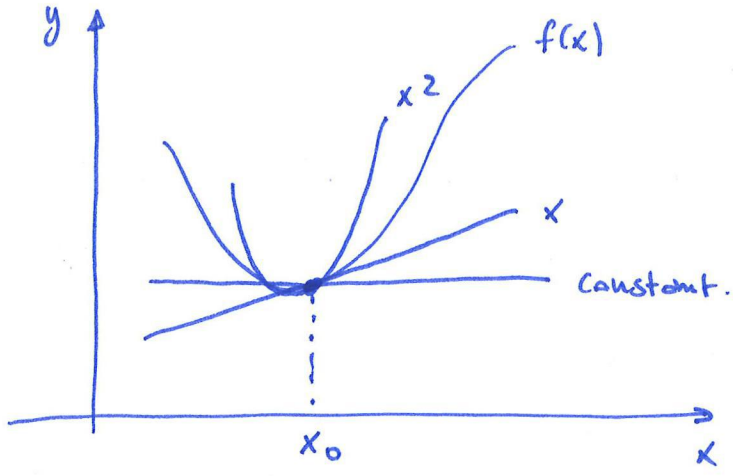
Interpolate data
- (Taylor) ~only useful for a single point.
- Lagrange
- Newton.
  - nested form
  - divided differences
  - Neville.

Function approx using poly interpol.
- (Error.)
- Better interpol nodes
- Splines. (lower order + local)

Taylor Series

* Taylor polynomial

"Find a polynomial p approximating a function f in a neighbourhood of a point x_0."



$$p_0(x) = a_0$$

$$p_1(x) = a_0 + a_1(x - x_0)$$

$$p_2(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2$$

$$\vdots$$

$$p_n(x) = \sum_{i=0}^{n} a_i (x - x_0)^i$$

(?) What coefficients should I use? Many choices.... suggestion: Let's match the derivatives of $p(x)$ and $f(x)$ at x_0.

$$p(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + a_3(x - x_0)^3 + \dots$$

$$p'(x) = a_1 + 2a_2(x - x_0) + 3a_3(x - x_0)^2 + \dots$$

$$p''(x) = 2a_2 + 6a_3(x - x_0) + \dots$$

$$p'''(x) = 6a_3 + \dots$$

$\hookrightarrow$ $p^{(k)}(x_0) = f^{(k)}(x_0) \iff k!\, a_k = f^{(k)}(x_0)$

$$\iff \boxed{a_k = \frac{f^{(k)}(x_0)}{k!}}$$

So: Taylor polynomial

$$p_n(x) = f(x_0) + f'(x_0)(x-x_0) + \frac{f''(x_0)}{2}(x-x_0)^2 + \frac{f'''(x_0)}{6}(x-x_0)^3 + \dots + \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n.$$

Example

$f(x) = e^x \cos(2x)$ at $x_0 = 0$ by $p_3(x)$.

↳ $p_3(x) = f(x_0) + f'(x_0)(x-x_0) + f''(x_0)(x-x_0)^2 + f'''(x_0)(x-x_0)^3$

$= f(0) + f'(0)x + \frac{f''(0)}{2}x^2 + \frac{f'''(0)}{6}x^3$

↳ $f(x) = e^x \cos(2x)$ | $f(0) = 1$

$f'(x) = e^x(\cos(2x) - 2\sin(2x))$ | $f'(0) = 1$

$f''(x) = e^x(-3\cos(2x) - 4\sin(2x))$ | $f''(0) = -3$

$f'''(x) = e^x(-11\cos(2x) + 2\sin(2x))$. | $f'''(0) = -11$

↳ $p_3(x) = 1 + x - \frac{3}{2}x^2 - \frac{11}{6}x^3$

→ see matlab

Taylor series = infinite sum.

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k$$

eg: $f(x) = e^x$ at $x_0 = 0$

$f^{(k)}(x) = e^x$ $f^{(k)}(0) = 1$.

so $e^x = \sum_{k=0}^{\infty} \frac{1}{k!}x^k$.

eg. $f(x) = \frac{1}{1-x} = (1-x)^{-1}$

$f^{(n)}(x) = k!(1-x)^{-(k+1)}$

so: $\frac{1}{1-x} = \sum_{k=0}^{\infty} x^k$ if $|x|<1$.

! Analytic functions: e^x, $\log(1+x)$, $\sin x$, $\cos x$, ...

* Error

$f(x) \approx p_n(x)$

↳ how big is this?

note: series = exact (∞)

polynomial = approx. (n)

Taylor's theorem If f is $(n+1)$-differentiable, then there exists $\xi(x)$ between x_0 and x such that.

$$f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k + \underbrace{\frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x-x_0)^{n+1}}_{\text{error term}}$$

(due to throwing away $n+1$ up to ∞.)

This gives us an error bound:

If $x, x_0 \in [a,b]$, then

$$|f(x) - p_n(x)| \leq \frac{1}{(n+1)!} \cdot \max_{\xi \in [a,b]} |f^{(n+1)}(\xi)| \cdot |x-x_0|^{n+1}$$

! This is the nice thing about Taylor polynomials! We can fully characterize the error!

? Suppose $f(x) = x^3 + 2x - 5$. What is the error for $p_2(x)$?

→ 0 because $f^{(4)}(x) = 0$

* Derivation for error (non-examinable), see extra notes.

* example: Approximate $e^{1/2}$ using $P_3(x)$ and $x_0 = 0$.

Taylor's theorem tells us that.

$$e^x = \underbrace{1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3}_{P_3(x)} + \underbrace{\frac{1}{24}e^{\xi}x^4}_{\text{error term}} \quad \text{for some } \xi \in (0, x).$$

↓

Can we bound this somehow? So that it's more useful... sometimes we can!

↳ note:

$e = 2{,}718 < 4$

$e^{1/2} < 4^{1/2}$

$e^{1/2} < 2$

↳ Also, e^x is monotonic.

↳ so, if $\xi \in (0, 1/2)$, then $e^{\xi} \in (e^0, e^{1/2}) \subset [1, 2]$

So: $e^{1/2} = 1 + \frac{1}{2} + \frac{1}{2}\cdot\frac{1}{4} + \frac{1}{6}\cdot\frac{1}{8} + \frac{1}{24}\cdot\frac{1}{16}\cdot[1,2]$

$= [1{,}6484375\ ;\ 1{,}6510416]$

$= 1{,}6497 \pm 0{,}0014$ → super nice!

cf.

exact value: 1,64872127 (8dp) within computed bounds

BUT Taylor provides a nice approx in the neighbourhood of a point, but not far away from it

↳ which is fine if you always work around a specific point (ct. engineering apps, optim.)

we have multiple data points

we need something else.

Error in Linear Taylor polynomial

1) Given a function $f(x)$ and a first-order Taylor approx $p(x) = f(x_0) + f'(x_0)(x - x_0)$ around x_0.

2) What is the error at the point x_*?
Define: $E_* = f(x_*) - p(x_*)$.

Can we find an expression for E_*? --- Yes.

3) Define a new aux. function.

$$q(x) = p(x) + \frac{E_* (x - x_0)^2}{(x_* - x_0)^2}.$$

this function will help us to characterize the error!
→ why ()²? we don't want to change $p(x)$ (first order, x)
→ why this particular form?

$q(x_0) = p(x_0) + 0$ → correct, because dw around x_0.

$q(x_*) = p(x_*) + E_*$ → poly + error in x_*.

4) Define a new function

$$\begin{aligned} g(x) &= q(x) - f(x). \\ &= p(x) + \frac{E_* (x - x_0)^2}{(x_* - x_0)^2} - f(x). \\ &= f(x_0) + f'(x_0)(x - x_0) + \frac{E_* (x - x_0)^2}{(x_* - x_0)^2} - f(x). \end{aligned}$$

note: $g(x_0) = f(x_0) + 0 + 0 - f(x_0) = 0$

$$\begin{aligned} g(x_*) &= p(x_*) + E_* - f(x_*) \\ &= E_* - \underbrace{(f(x_*) - p(x_*))}_{= E_*} = 0. \end{aligned}$$

To conclude: we did all of this effort so we can apply Rolle (typical switch in calculus)

4) Apply Rolle to $g(x)$.

$$g(x_0) = 0$$
$$g(x_*) = 0$$

$$\rightarrow \exists\, \xi_1 \in (x_0, x_*) : g'(\xi_1) = 0.$$

$$g'(x) = f'(x_0) + 2E_* \frac{(x - x_0)}{(x - x_0)^2} - f'(x)$$

$$! \quad g'(x_0) = 0.$$

5) Apply Rolle to $g'(x)$

$$\rightarrow \exists\, \xi_2 \in (x_0, \xi_1) : g''(\xi_2) = 0$$

$$g''(x) = \frac{2E_*}{(x_* - x_0)^2} - f''(x)$$

$$\exists\, \xi_2 : \; g''(\xi_2) = 0 \iff \frac{2E_*}{(x_* - x_0)^2} - f''(\xi_2) = 0.$$

$$\iff \boxed{E_* = \frac{1}{2} f''(\xi_2)(x_* - x_0)^2}$$

6) $f(x) \approx p(x)$

$$f(x) = p(x) + E^* \frac{(x - x_0)^2}{(x_* - x_0)^2}$$

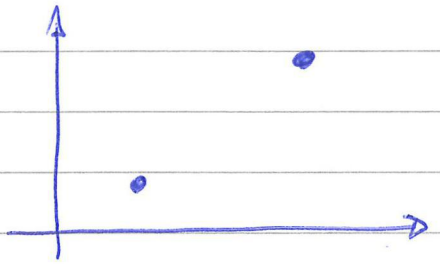
$$= p(x) + \frac{1}{2} f''(\xi)(x - x_0)^2 \qquad * \; \xi \in (x_0, x^*)$$

nice because
$|f(x) - p(x)| \leq \ldots$
if $f'' \approx 0$, then good approx.
makes sense, since nearly lin. func.

recap slide 17
slides 13 + 18 → do in Matlab

Lagrange polynomials

Consider $N=2$: (x_0, y_0) , (x_1, y_1).



Want a first degree poly (i.e. lin func) through these points

i.e. $p(x_i) = y_i$

One way to do that is as follows.

$$p(x) = \frac{x - x_1}{x_0 - x_1} \cdot y_0 + \frac{x - x_0}{x_1 - x_0} \cdot y_1.$$

if $x = x_0$: $p(x_0) = \underbrace{\frac{x_0 - x_1}{x_0 - x_1}}_{1} \cdot y_0 + \underbrace{\frac{x_0 - x_0}{x_1 - x_0}}_{0} y_1.$

$$p(x_0) = y_0 \quad \checkmark$$

if $x = x_1$: $p(x_1) = \underbrace{\frac{x_1 - x_1}{x_0 - x_1}}_{0} \cdot y_0 + \underbrace{\frac{x_1 - x_0}{x_1 - x_0}}_{1} \cdot y_1$

$$p(x_1) = y_1 \quad \checkmark$$

This is quite nice because I can construct everything based on | data points. using simple formulas.
| interpol nodes

e.g. Construct Lagrange poly of degree 1 through $(2,4)$ and $(5,1)$. ($x_0\ y_0$ and $x_1\ y_1$)

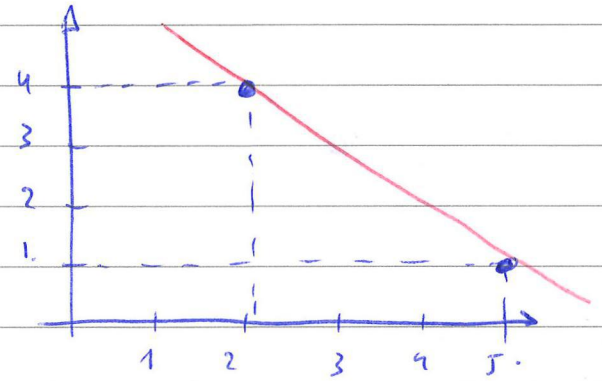
$$p(x) = \frac{x-x_1}{x_0-x_1}\cdot y_0 + \frac{x-x_0}{x_1-x_0} y_1$$

$$= \frac{x-5}{2-5}\cdot 4 + \frac{x-2}{5-2}\cdot 1$$

$$= -\frac{1}{3}(x-5)\cdot 4 + \frac{1}{3}(x-2)\cdot 1$$

$$= \dots$$

$$= -x+6.$$



! Same as std linear fct of course, see further (just show it)

↳ Let's generalize

$$p(x) = \ell_0(x)\cdot y_0 + \ell_1(x) y_1.$$

$\ell_0(x)$, $\ell_1(x)$ → Lagrange basis functions

↓

draw them, see slides.

N=3: $(x_0, y_0), (x_1, y_1), (x_2, y_2)$.

$$p(x) = \underbrace{l_0(x)}y_0 + \underbrace{l_1(x)}y_1 + \underbrace{l_2(x)}y_2$$

↘ ↓ ↙
?

$$p(x) = \underbrace{\left(\frac{x-x_1}{x_0-x_1}\right)\left(\frac{x-x_2}{x_0-x_2}\right)}_{l_0(x)} y_0 + \underbrace{\left(\frac{x-x_0}{x_1-x_0}\right)\left(\frac{x-x_2}{x_1-x_2}\right)}_{l_1(x)} y_1 + \underbrace{\left(\frac{x-x_0}{x_2-x_0}\right)\left(\frac{x-x_1}{x_2-x_1}\right)}_{l_2(x)} \cdot y_2.$$

if $x = x_0$: $p(x_0) = \underbrace{\left(\frac{x_0-x_1}{x_0-x_1}\right)}_{1}\underbrace{\left(\frac{x_0-x_2}{x_0-x_2}\right)}_{1} y_0 + 0\cdot(\ )\cdot y_1 + 0\cdot(\ )\cdot y_2$

$p(x_0) = y_0$.

⋮

N+1 $(x_0, y_0) \quad \dots \quad (x_N, y_N)$.

i is left out

define: $l_i(x) = \frac{(x-x_0)}{(x_i-x_0)} \cdots \frac{(x-x_{i-1})}{(x_i-x_{i-1})} \frac{(x-x_{i+1})}{(x_i-x_{i+1})} \cdots \frac{(x-x_n)}{(x_i-x_n)}$

$$= \prod_{\substack{j\neq i \\ j=0}}^{N} \left(\frac{x-x_j}{x_i-x_j}\right)$$ general lagrange basis.

→ $l_i(x_j) = 1$ if $i = j$.
$l_i(x_j) = 0$ if $i \neq j$.

$$p(x) = \sum_{i=0}^{n} l_i(x) \cdot y_i$$

→ example (slides) 23-24.

existence part is proved!

Existence & uniqueness

* We can always do this (by construction) → existence part is proved!

$$p(x) = \sum_{i=0}^{n} l_i(x) \cdot y_i \xrightarrow[\text{simplify}]{\text{expand \&}} p(x) = \sum_{i=0}^{n} c_i x^i.$$

e.g. first degree example from before!

what about uniq.?

* we have n+1 linear equations (in c_i).

$$c_0 + c_1 x_i + c_2 x_i^2 + \cdots + c_n x_i^n = y_i.$$

* write as matrix (n+1 × n+1)

$$\underbrace{\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & & & & \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix}}_{=V} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Show loC of polyfit. "Algorithms" now we know what's happening when you call polyfit.

* Does a sol to this system exist AND is unique? (yes for all data, if diff nodes).

1) This is a Vdm matrix with $\det(V) \neq 0$ iff $x_i \neq x_j$ if $i \neq j$. (all distinct nodes).

$\det V = \prod_{i<j} (x_j - x_i)$ of ordered. for all $x_i \neq x_i$

2) If $\det V \neq 0$ → V has full (col.) rank n+1. (i.e. the nullity is zero) → Unique sol.

3) by 1) & 2), unique sol.

mention (A) and (B) page 10

$$y = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1$$

$$= -\frac{x - x_1}{x_1 - x_0} y_0 + \frac{x - x_0}{x_1 - x_0} y_1$$

$$= \frac{-xy_0 + x_1 y_0 + xy_1 - x_0 y_1 + x_0 y_0 - x_0 y_0}{x_1 - x_0}$$

$$= \frac{(y_1 - y_0)}{(x_1 - x_0)} \cdot x - \frac{(y_1 - y_0)}{(x_1 - x_0)} x_0 + \frac{(x_1 - x_0) y_0}{(x_1 - x_0)}$$

$$\boxed{y = \left(\frac{y_1 - y_0}{x_1 - x_0}\right) \cdot x + y_0}$$

standard linear fit formula.

Newton polynomial interpolation

Explain via an example.

x_i	1	3	-2	(4
y_i	5,0	1,0	-4,0	9,5)

Idea: Interpolate step by step (instead of all-at-once as 2!)

* zero-degree poly through (1, 5, 0):

$$p_0(x) = a_0$$
$$p_0(1) = a_0 = 5,0.$$

* first-degree poly through (1; 5,0) AND (3; 1,0) WITHOUT CHANGING THE VALUE AT x_0. !

$$p_1(x) = p_0(x) + (x - x_0)\, a_1$$
$$= a_0 + (x - x_0)\, a_1$$

↳ this works! $p_1(x_0) = a_0 + (x_0 - x_0)\, a_1 = a_0.$
↳ Find a_1 (since $x_0 - x_0 = 0$)

$$p_1(x_1) = 5,0 + (3-1)\cdot a_1 = y_1 = 1.$$
$$\dots a_1 = -2,0.$$

$$p_1(x) = 5,0 + (x - 1,0)(-2,0)$$

* 2nd-degree poly through (1; 5,0) AND (3; 1,0) AND (-2, -4, 0)
WITHOUT CHANGING THE VALUE AT x_0 AND x_1

$$p_2(x) = p_1(x) + (x - x_0)(x - x_1)\, a_2.$$

↳ this works!

$$p_2(x_0) = p_1(x_0) + (x_0 - x_0)(x_0 - x_1)\, a_2 \quad (= a_0.$$

$p_1(x_0) = a_0$ (see before)

$$p_2(x_1) = p_1(x_1) + (x_1 - x_0)(x_1 - x_1)\, a_2 \quad (=$$

$p_1(x_1) = 1$ (see before)

↳ Find a_2:

$$p_2(x_2) = p_1(x_2) + (x_2 - x_0)(x_2 - x_1)\, a_2 = y_2$$

$$\dots \quad a_2 = -1.$$

etc ... ☒ Strongly recommend to do the next steps yourself by hand!

Note: Nested Form ~ Horner's method

$$\begin{aligned} p_2(x) &= p_1(x) + (x - x_0)(x - x_1)\, a_2 \\ &= a_0 + (x - x_0) a_1 + (x - x_0)(x - x_1)\, a_2 \\ &= a_0 + (x - x_0)(a_1 + (x - x_1) a_2) \\ &= 5 + (x-1)(-2 + (x-3)(-1)) \end{aligned}$$

Nested eval

vs

$$\begin{aligned} p_2(x) &= 5 + (x-1)(-2 - x + 3) \\ &= 5 + (x-1)(1-x) \\ &= 5 + x - x^2 - 1 + x \\ &= -1x^2 + 2x + 4 \end{aligned}$$

~ see lecture 1.

direct eval

→ test it with 3 digit arithmetic.

(▽) Nice pop: If you have an extra datapoint, you can simply add (cf. Lagrange, where you have to redo everything! But you can precompute vs on the fly!

General case

$$p_0(x) = a_0$$

$$p_1(x) = a_0 + (x - x_0)\, a_1$$

$$p_2(x) = a_0 + (x - x_0)\, a_1 + (x - x_0)(x - x_1)\, a_2$$

$$= a_0 + (x - x_0)(a_1 + (x - x_1)\, a_2)$$

$$p_3(x) = a_0 + (x - x_0)\, a_1 + (x - x_0)(x - x_1)\, a_2 + (x - x_0)(x - x_1)(x - x_2)\, a_3$$

$$= a_0 + (x - x_0)(a_1 + (x - x_1)(a_2 + (x - x_2)\, a_3))$$

$$\vdots$$

$$p(x) = a_0 + (x - x_0)(a_1 + \cdots + (x - x_{n-2})(a_{n-1} + (x - x_{n-1})\, a_n) \cdots)$$

$$= \sum_{i=0}^{n} a_i \prod_{j=0}^{i-1} (x - x_j)$$

~ det in compact but opaque because it doesn't illustrate the recursive prop.

Recursive eval ....

$p = a_n$

for $i = n-1 : -1 : 1$

$p = a_i + \text{aux}\, p \cdot (x_0 - x_i)$

end

Neville's method ---

→ Another way to compute Newton interp. poly.

* First, note the following.

Suppose $P[f; x_0, \dots, x_k]$ is an interpol poly of f at $x_0 \dots x_k$.

Suppose the nested form of the interp poly at $x_0 \dots x_n$ is

$$P[f; x_0, \dots, x_n] = \sum_{i=0}^{n} a_i \prod_{j=0}^{i-1} (x - x_j).$$

then for $k < n$, the interpol poly at $x_0 \dots x_k$ has the same coeff!

$$P[f; x_0, \dots, x_k] = \sum_{i=0}^{k} a_i \prod_{j=0}^{i-1} (x - x_j)$$

example from before

$$P_0[f; x_0] = 5,0 \qquad = p_0(x)$$

$$P[f; x_0, x_1] = 5,0 + (x - 1,0)(-2,0) \qquad = p_1(x)$$

$$P[f; x_0, x_1, x_2] = 5,0 + (x - 1,0)(-2,0 + (x-3)(-1)) \qquad = p_2(x)$$

* Neville's method uses the following formula based on that fact.

$$P[f; x_0 \dots x_k] = \frac{(x - x_0) P[f; x_1 \dots x_k](x) - (x - x_k) P[f; x_0 \dots x_{k-1}](x)}{x_k - x_0}.$$

~> you don't need to know the derivation; only if it makes sense.

How? check if $P[f; x_0 \dots x_k](x_0) \stackrel{?}{=} f(x_0)$
$(x_1) \stackrel{?}{=} f(x_1)$ — Try it!

x_i	1	3	-2
y_i	5,0	1,0	-4,0

$P[f;x_0] = 5,0$
$P[f;x_1] = 1,0$
$P[f;x_2] = -4,0$

$$P[f;x_0,x_1] = \frac{(x-x_0)P[f;x_1] - (x-x_1)P[f;x_0]}{x_1-x_0}$$

$$= \frac{(x-1)\cdot 1,0 - (x-3)\cdot 5,0}{3-1}$$

$$= \frac{x-1-5x+15}{2}$$

$$= \frac{-4x+14}{2}$$

$$= -2x+7. \quad = 5,0 + (x-1,0)\cdot(-2,0)$$

$$P[f;x_1,x_2] = ((x-x_1)P[f;x_2] - (x-x_2)P[f;x_1])/(x_2-x_1)$$
$$= ((x-3)(-4,0) - (x--2)\cdot 1,0)/(-2-3)$$
$$= (-4x+12-x-2)/(-5)$$
$$= (-5x+10)/(-5)$$
$$= x-2$$

$$P[f;x_0,x_1,x_2] = ((x-x_0)\cdot P[f;x_1,x_2] - (x-x_2)\cdot P[f;x_0,x_1])/(x_2-x_0)$$
$$= ((x-1)(x-2) - (x+2)(-2x+7))/(-2-1)$$
$$= (x^2-2x-x+2-(-2x^2+7x-4x+14))/(-3)$$
$$= (3x^2-6x-12)/(-3)$$
$$= -x^2+2x+4 \quad = 5+(x-1)(-2)+(x-3)(-1)$$

Divided diffs = final method, and "best" method, to compute Newton interpol poly.
(in numerical sense. (for the three that we see))

* Nested form of interpol poly through $x_0 \cdots x_k$.

$$p[f; x_0, \ldots, x_k](x) = a_0 + (x - x_0)(a_1 + \cdots + (x - x_{k-2})(a_{k-1} + (x - x_{k-1})a_k) \cdots)$$

The coefficient of x^k in a_k. (if you explicitize).

* Let's denote/write the coeff of x^k in $p[f; x_0 \cdots x_k]$ as.

$$f[x_0, \ldots, x_k].$$

then we see that $a_k = f[x_0, \ldots, x_k]$.

* Neville's method says/uses:

$$p[f; x_0, \ldots, x_k](x) = \frac{(x - x_0)\, p[f; x_1, \ldots, x_k](x) - (x - x_k)\, p[f; x_0, \ldots, x_{k-1}](x)}{x_k - x_0}$$

If we only look at the coefficient of x^k, then we have

$$f[x_0, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}$$

we call this divided difference

In words: the coeff of x^k in the interpol poly through $x_0 \cdots x_k$ is equal to the diff between the coeff of the interpol poly through $x_1 \cdots x_k$ and the coeff of the interpol poly through $x_0 \cdots x_{k-1}$, divided by $x_k - x_0$.

▽ Note: Neville : we compute the full polys.

Div diff : we only compute the coeffs, needed for nested form

Let's check for previous example.

$p_0(x) = 1$

$p_1(x) = 1 + (x-3)a_1$

$p_1(x_2) = -4 = 1 + (-2-3)a_1 \Leftrightarrow 1 - 5a_1 = -4$

$a_1 = 1.$

Same approach as before.

$P[f; x_1, x_2] = 1 + (x-3)(1).$

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

$P[f; x_0, x_1] = 5 + (x-1)(-2).$

$x_2 - x_0 \rightarrow -2 - 1.$

$$= \frac{1 - (-2)}{-2 - 1}$$

$$= -1$$

$P[f; x_0, x_1, x_2] = (-1)x^2 + 2x + 4.$

How to show formula? Full def. div diff.

$$f[x_i] = f(x_i) \quad (= \text{coeff of } x^0 \text{ term})$$

$$f[x_i \cdots x_j] = \frac{f[x_{i+1} \cdots x_j] - f[x_i \cdots x_{j-1}]}{x_j - x_i}$$

↓

Example in slides. ~~38~~ 33 OR better

40 + formulas in 37.

01 example next page

mention maps: 41

Newton: (cont'd example)

$f[x_0] = f(x_0) = 5,0$
$f[x_1] = f(x_1) = 1,0$
$f[x_2] = f(x_2) = -4,0.$

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{1 - 5}{3 - 1} = -2$$

$f[x_1, x_2] = \ldots = 1$

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{1 - -2}{-2 - 1} = \frac{3}{-3} = -1$$

→ Good method.

Divided differences

Example Interpolate data

x_i	1	-4	0
$f(x_i)$	0.3	1.3	-2.3

Nested form: $p(x) = 0.3 + (x - 1) \times (-0.2 + (x + 4) \times 0.7)$.

Check:

$$p(1) = 0.3 + (1-1) \times (-0.2 + (1 + 4) \times 0.7) = 0.3+0\times(\cdots) = 0.3.$$
$$p(-4) = 0.3 + (-4-1) \times (-0.2 + (-4+4) \times 0.7)$$
$$= 0.3-5\times(-0.2+0\times0.7) = 1.3.$$
$$p(0) = 0.3 + (0-1) \times (-0.2 + (0+4) \times 0.7)$$
$$= 0.3 - (-0.2 + 4 \times 0.7) = 0.3 - 2.6 = -2.3.$$

Note: Only the check $p(x_n) = y_n$ at the final data point tests all calculated coefficients!

Note: The expanded form is $p(x) = 0.7x^2 + 1.9x - 2.3$, but nested form is usually more accurate to evaluate, so it is better to leave your answer in nested form.

39 / 73

Divided differences

Example Interpolate data

x_i	1	2	4	5
$y_i = f(x_i)$	3	1	2	6

Divided differences

$x_0 = 1$	$f[x_0] = 3$	$f[x_0, x_1] = -2$	$f[x_0, x_1, x_2] = 5/6$	$f[x_0, x_1, x_2, x_3] = 1/12$
$x_1 = 2$	$f[x_1] = 1$	$f[x_1, x_2] = 1/2$	$f[x_1, x_2, x_3] = 7/6$	
$x_2 = 4$	$f[x_2] = 2$	$f[x_2, x_3] = 4$		
$x_3 = 5$	$f[x_3] = 6$			

Interpolating polynomial

$$p(x) = 3 + (x - 1) \times \left(-2 + (x - 2) \times \left(\tfrac{5}{6} + (x - 4) \times \tfrac{1}{12}\right)\right).$$

Check by computing $p(x)$ at interpolation point x_3.

$$p(x_3) = 3 + (5 - 1) \times \left(-2 + (5 - 2) \times \left(\tfrac{5}{6} + (5 - 4) \times \tfrac{1}{12}\right)\right)$$
$$= 3 + 4 \times \left(-2 + 3 \times \left(\tfrac{5}{6} + 1 \times \tfrac{1}{12}\right)\right) = 3 + 4 \times \left(-2 + 3 \times \tfrac{11}{12}\right)$$
$$= 3 + 4 \times \left(-2 + \tfrac{11}{4}\right) = 3 + 4 \times \tfrac{3}{4} = 3 + 3 = 6 = f(x_3).$$

40 / 73

Properties of divided differences

Symmetry The divided difference $f[x_0, \ldots, x_k]$ is independent of the order of the variables:

$$f[x_0, \ldots, x_i, \ldots, x_j, \ldots, x_k] = f[x_0, \ldots, x_j, \ldots, x_i, \ldots, x_k].$$

e.g. $f[x_0, x_1, x_2, x_3] = f[x_0, x_3, x_2, x_1]$; $f[x_1, x_2, x_4] = f[x_4, x_1, x_2]$.

Order of computation The divided differences can be computed in many ways.

$$f[x_0, x_1, x_2, x_3] := \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{f[x_0, x_2, x_3] - f[x_1, x_2, x_3]}{x_0 - x_1}.$$

Explicit formula From the Lagrange form of the interpolating polynomial:

$$f[x_0, \ldots, x_k] = \sum_{i=0}^{k} \frac{f(x_i)}{\prod_{j=0; j\neq i}^{k} (x_i - x_j)}.$$

41 / 73

Extended divided differences (Non-Examinable)

Theorem (Divided differences and derivatives) If $f^{(n)}$ is continous on $[a, b]$ and $x_0, \ldots, x_n$ are distinct points in $[a, b]$, then there exists $\xi \in [a, b]$ such that $f[x_0, \ldots, x_n] = f^{(n)}(\xi)/n!$.

i.e. the n^{th} divided differences are approximations to $f^{(n)}(x)/n!$

Extended divided differences Extend divided differences to the case some of the x_i are equal by defining

$$f[x, x] = f'(x); \quad f[x, x, x] = f''(x)/2; \quad f[x, x, \ldots, x] = f^{(n)}(x)/n!$$

Then we can compute e.g.

$$f[x, x, y] = \frac{f[x, y] - f[x, x]}{y - x} = \frac{\frac{f(y)-f(x)}{y-x} - f'(x)}{y - x}.$$

Newton's nested form extends naturally to this case!

42 / 73

Function Approximation by Polynomial Interpolation

43 / 73

Function approximation

Function Approximation Given a function $f : [a, b] \to \mathbb{R}$, find a function g such that $g(x) \approx f(x)$

Approximation error Minimise the *uniform/supremum norm*

$$||f - g||_\infty := \sup_{x\in[a,b]} |f(x) - g(x)|.$$

or (easier) the *two-norm*

$$||f - g||_2 := \left(\int_a^b |f(x) - g(x)|^2 \, dx\right)^{1/2}.$$

Approximation by interpolation Often compute g as a function interpolating f at points $x_0, x_1, \ldots, x_n$.

Applications

- Often used for computer arithmetic, by approximating a transcendental function (such as $\exp(x)$) by polynomial or rational function.
- May also be used to *simplify* a model by replacing a slow-to-evaluate function by a faster-to-evaluate approximation.

Error of polynomial interpolation

Lagrange basis example Interpolate $y(0) = 1, y(i) = 0$ for $i = -m, \ldots, +m$.

```
m=4, n=2*m; xs=[-m:+m], ys=zeros(1,n+1); ys(m+1)=1,
cs = polyfit(xs,ys,n), p = @(x)polyval(cs,x),
p(xs),
plot([-m,+m],[0,0]); hold; fplot(p,[-m,+m]); hold;
```

Runge example Interpolate $f(x) = 1/(1+x^2)$ using $n+1$ equally spaced nodes on $[-4, +4]$.

```
f=@(x)1./(1+x.^2); a=4;
n=8, xs=linspace(-a,+a,n+1), ys=f(xs),
cs = polyfit(xs,ys,n); p = @(x)polyval(cs,x),
fplot(f,[-a,+a]); hold; fplot(p,[-a,+a]); hold;
```

Errors $e_8 = 0.73, e_{16} = 5.9, e_{32} = 7.1 \cdot 10^2, e_{64} = 2.8 \cdot 10^8$.

Approximation accuracy worsens as n increases!!

Error of interpolating polynomial

Theorem (Error of interpolating polynomial) If p is the polynomial of degree at most n that interpolates f at the $n+1$ distinct nodes $x_0, x_1, \ldots, x_n$ belonging to an interval $[a, b]$, and if $f^{(n+1)}$ is continuous, then for each x in $[a, b]$, there is a ξ in (a, b) for which

$$f(x) - p(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{i=0}^{n} (x - x_i)$$

Hence

$$|f(x) - p(x)| \leq \frac{1}{(n+1)!} \max_{\xi \in [a,b]} |f^{(n+1)}(\xi)| \prod_{i=0}^{n} |x - x_i|$$

Error of interpolating polynomial

Proof of interpolating polynomial error (Non-Examinable)

Let p interpolate f at $x_0, \ldots, x_n$. Fix x_*. Let $E_* = f(x_*) - p(x_*)$.

Let l_* be the Lagrange polynomial $l_*(x_*) = 1$ and $l_*(x_i) = 0$ for $i = 0, \ldots, n$.

Then $p(x_i) + E_* l_*(x_i) = f(x_i)$ and $p(x_*) + E_* l_*(x_*) = f(x_*)$.

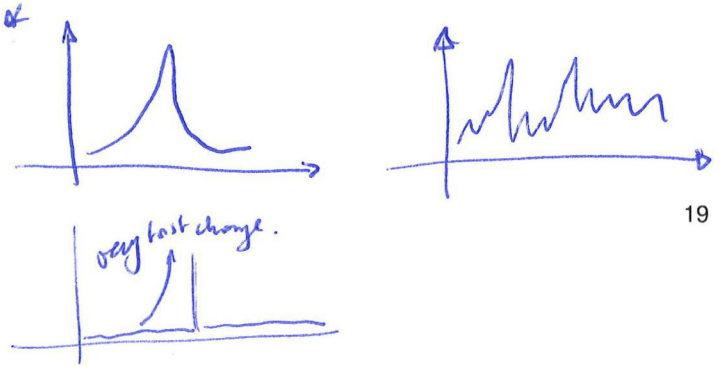
Let $g(x) = p(x) + E_* l_*(x) - f(x)$ which has zeros at $x_0, \ldots, x_n, x_*$.

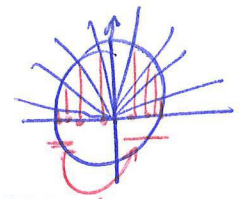
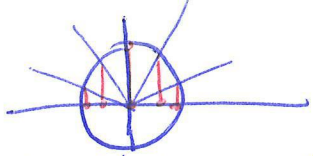
By Rolle's theorem, there exists ξ such that $g^{(n+1)}(\xi) = 0$.

Note for all x, $p^{(n+1)}(x) = 0$ and $l_*^{(n+1)}(x) = (n+1)! / \prod_{i=0}^{n}(x_* - x_i)$.

Hence $E_*(n+1)! / \prod_{i=0}^{n}(x_* - x_i) - f^{(n+1)}(\xi) = 0$.

Rearranging gives $f(x_*) - p(x_*) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{i=0}^{n}(x_* - x_i)$.



visualizer

use better nodes.

more dense in the end points.

range between −1 & 1 scaled over [0,5].

Chebyshev nodes

Chebyshev nodes Interpolation over $[a, b]$ often best using nodes

$$x_k = \frac{a+b}{2} - \frac{b-a}{2}\cos\left(\frac{2k+1}{2(n+1)}\pi\right) \text{ for } k = 0, \ldots, n.$$

Runge example Interpolate $f(x) = 1/(1+x^2)$ using $n+1$ Chebyshev nodes on $[-4, +4]$.

```
f=@(x)1./(1+x.^2); a=4;
n=8, xs=-a*cos((2*[0:n]+1)*pi/(2*n+2)), ys=f(xs),
cs = polyfit(xs,ys,n); p = @(x)polyval(cs,x);
fplot(f,[-a,+a]); hold on; fplot(p,[-a,+a]); hold off;
```

Matlab Script 6

Errors $e_8 = 0.10$, $e_{16} = 0.015$, $e_{32} = 2.8 \cdot 10^{-4}$.

Approximation accuracy improves as n increases.

Error of interpolating polynomials

Theorem (Interpolation error with equally-spaced nodes) If $p(x)$ is the interpolating polynomial of f with $n+1$ equally-spaced nodes on $[a, b]$, then

$$|f(x) - p(x)| \leq \frac{(b-a)^{n+1}}{4n^{n+1}(n+1)} \max_{x\in[a,b]} |f^{(n+1)}(\xi)|$$

Theorem (Interpolation error with Chebyshev nodes) If $p(x)$ is the interpolating polynomial of f with $n+1$ Chebyshev nodes, then

$$|f(x) - p(x)| \leq \frac{(b-a)^{n+1}}{2^{2n+1}(n+1)!} \max_{x\in[a,b]} |f^{(n+1)}(\xi)|$$

Example If $f(x) = \sin(x)$ on $[-1, +1]$, then $\max_{x\in[a,b]} |f^{(n+1)}(\xi)| \leq 1$.
Hence with $4+1$ equally-spaced nodes have error

$$\epsilon \leq 2^{n+1}/4n^{n+1}(n+1) = 2^5/(4 \cdot 4^5 \cdot 5) = 1/640$$

and with $4+1$ Chebyshev nodes,

$$\epsilon \leq 2^{n+1}/2^{2n+1}(n+1)! = 1/(2^4 \cdot 5!) = 1/1920.$$

there exist problems where even chebychev nodes can't help us, but we have theorems for them.

Approximation theorems (Non-examinable)

we can find poly's with same bound.

1

Theorem (Weierstrass) Let f be continuous on $[a, b]$. Then for every $\epsilon > 0$, there exists a polynomial p such that

$$||f - p||_\infty := \sup_{x\in[a,b]} |f(x) - p(x)| < \epsilon.$$

2

Theorem (Chebyshev alternation) Let f be continuous on $[a, b]$. Then there is a unique best approximating polynomial p_m of degree m.

Further, there exist $m+2$ points $w_0, \ldots, w_{m+1}$ such that

$$f(w_i) - p_m(w_i) = \pm(-1)^i ||f - p_m||_\infty,$$

and $m+1$ points $x_0, \ldots, x_m$ such that $p_m(x_i) = f(x_i)$.

3

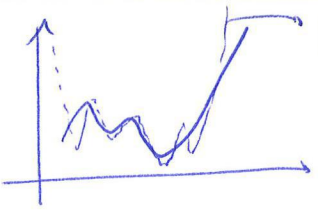
If f is n-times continuously differentiable, there is a constant C such that $||f - p_m|| \leq C/m^n$, and if f is smooth (*analytic*), then there is are constants C and $R > 1$ such that $||f - p_m|| \leq C/R^m$.

is equal to its power series

4

The best approximating polynomial can be computed using the *Remez exchange algorithm*. → for the interested student.

this thm tells us that such points exist. however not always clear how to find those points



poly approx over entire interval typically.
leads to Runge's phenomenon: func vals ↑ at edges
+ oscillatory behavior of polynomials
due to HIGH ORDER

global vs local

Spline Interpolation

IDEA: Approx the func in segments with lower order polys.

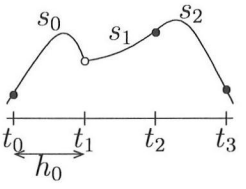
51 / 73

Splines

"piecewise" poly interpol aka spline interpol.

→ ex: see fig. 3.12 & 3.13, p. 96

Definition A *spline* of degree n on $[a, b]$ with *knots* at $a = t_0 < t_1 < \cdots < t_{n+1} = b$ is an function s such that s is equal to a degree-n polynomial s_i on $[t_i, t_{i+1}]$ and s is $n-1$ times differentiable (with continuous derivative) at each t_i.



The function above is not a cubic spline since it is not differentiable at t_1.

$$\lim_{x \nearrow t_1} s'(x) = s_0'(t_1) \neq s_1'(t_1) = \lim_{x \searrow t_1} s'(x)$$

52 / 73

Spline interpolation

(Cubic) Spline Interpolation Compute a (cubic) spline s such that $s(x_i) = y_i$ for $i = 0, \ldots, n$.

Knots at interpolation points For cubic splines, take knots at interpolation points, so $t_i = x_i$ for $i = 0, \ldots, n$.

53 / 73

Spline Interpolation in Matlab

Computing splines The command

```
S=spline(X,Y)
```

computes the spline interpolating data

$$X = [x_1, x_2, \ldots, x_n],\ Y = [y_1, y_2, \ldots, y_n]$$

with *not-a-knot* end conditions $s'''(x_2) = s'''(x_{n-1}) = 0$.

The command

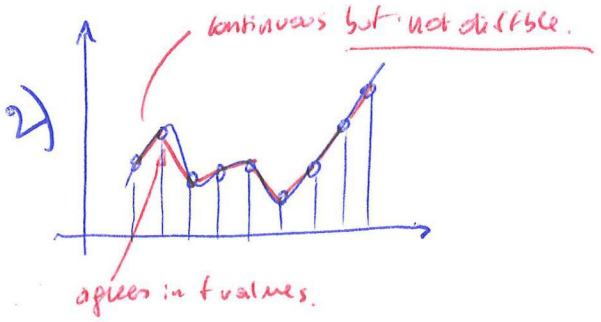
```
S=spline(X,[b1,Y,bn])
```

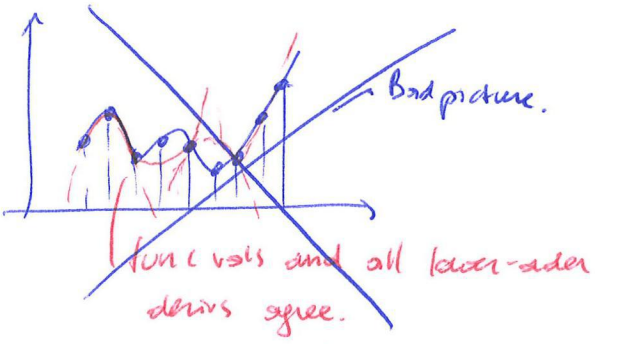
computes the spline interpolating data X, Y with *clamped* end conditions $s'(x_1) = b_1$ and $s'(x_n) = b_n$.

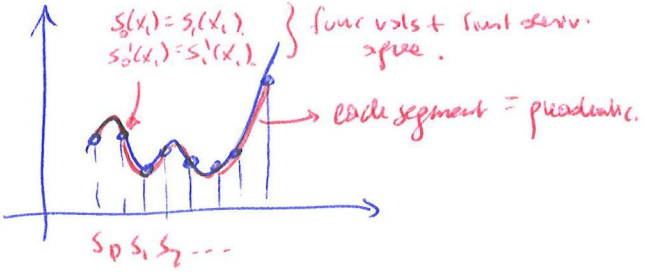
Evaluating splines To evaluate the spline S at x, use the command

```
ppval(S,x)
```

54 / 73







Spline Interpolation in Matlab

Example Standard basis $s(0) = 1$, otherwise $s(i) = 0$, for $i = -n, \ldots, n$.

```
n=4; X=[-n:+n], Y=0*X; Y(n+1)=1,
S=spline(X,Y), s=@(x) ppval(S,x),
plot([-n:+n],[0,0]); hold; fplot(s,[-n,+n]); hold
```

Example Interpolate $f(x) = 1/(1+x^2)$ on $[-4, 4]$ with clamped ends.

```
f=@(x)1./(1+x.^2), df=@(x) 2*x./(1+x.^2).^2, a=-4, b=+4,
n=8; X=linspace(a,b,n+1), Y=f(X), wa=df(a), wb=df(b),
S=spline(X,[wa,Y,wb]), s=@(x) ppval(S,x),
fplot(f,[-4,+4]); hold on; fplot(s,[-4,+4]); hold off;
```

Spline interpolation does not suffer from the extreme oscillations which may occur in polynomial interpolation!

55 / 73

→ get curve: illustrator. / ~~choosy (practical)~~ pen tool

Spline interpolation conditions (Non-examinable)

Spline formulae For $x \in [x_j, x_{j+1}]$, write $s(x) = s_j(x)$ given as

$$s_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.$$

The derivatives of s_j are

$$s_j'(x) = b_j + 2c_j(x - x_j) + 3d_j(x - x_j)^2,$$

$$s_j''(x) = 2c_j + 6d_j(x - x_j).$$

Knot points At knot points, $s(x_j) = s_j(x_j) = a_j$, $s'(x_j) = s_j'(x_j) = b_j$, $s''(x_j) = s_j''(x_j) = 2c_j$. Note $c_j = s''(x_j)/2$.

Interpolation conditions The interpolation conditions at x_j imply $a_j = y_j$ for $j = 0, \ldots, n$.

Continuity conditions The continuity of s, s', s'' imply for $j = 1, \ldots, n-1$

$$s_{j-1}(x_j) = s_j(x_j), \quad s_{j-1}'(x_j) = s_j'(x_j), \quad s_{j-1}''(x_j) = s_j''(x_j).$$

Alternatively, reindexing gives for $j = 0, \ldots, n-2$.

$$s_j(x_{j+1}) = s_{j+1}(x_{j+1}) \quad s_j'(x_{j+1}) = s_{j+1}'(x_{j+1}), \quad s_j''(x_{j+1}) = s_{j+1}''(x_{j+1}).$$

56 / 73